Fairway Dashboard

Development Time Record

Time clarification. I aimed to keep the hands-on development work within the six-hour limit. Based on my best reconstruction of the work, I finished at approximately six and a half hours, meaning I went about 15-30 minutes beyond the target. I want to be transparent about that overage. It occurred during final automated and mobile testing, when I investigated and corrected issues that were not apparent during the initial implementation. I chose to finish verifying those behaviors instead of stopping with known inconsistencies.

The development total below does not include preliminary planning, development-environment setup, GitHub repository organization, screenshots, written documentation, or the final requirements review. I completed those supporting activities separately so the submission would be organized and reproducible. I apologize if those activities were also expected to be included within the six-hour limit; my understanding was that the limit applied primarily to implementation. Nevertheless, the hands-on development and testing took approximately six and a half hours rather than exactly six.

Development timeline

Hour 0-1 | Application foundation and product data

During the first hour, I worked from my existing Next.js application framework and API code to establish the product dashboard's structure. I organized the application into routes for the product list, new-product form, product details, and product editing. I also defined the shared TypeScript product model and connected the catalog data to the interface.

- Organized the Next.js route structure.
- Connected the existing API and product-data logic.
- Defined reusable product types and categories.
- Created a catalog containing more than 15 products.
- Added unique product images.
- Built the initial responsive product listing.
- Displayed the required ID, name, category, price, stock, status, and thumbnail fields.

Hour 1-2 | Search, filters, sorting, and inventory views

During the second hour, I implemented the catalog controls. I added case-insensitive searching, category and status filters, and ascending and descending price sorting. I designed the filtering pipeline so the controls would work together regardless of the order in which they were applied.

- Added case-insensitive product-name searching.
- Added category and status filtering.
- Added ascending and descending price sorting.
- Combined search, filtering, and sorting behavior.
- Added filtered-result counts and clear-filter behavior.
- Added grid and table inventory views.
- Preserved appropriate filter state when switching views or categories.

Hour 2-3 | Product details, forms, and management actions

During the third hour, I completed the main product-management workflows. I implemented the dynamic /products/[id] route, built the add and edit forms, and connected successful submissions to the displayed inventory. I also added archive, restore, and delete functionality.

- Built the dynamic product-details page and used its route parameter to locate the selected product.
- Added a useful response for an unknown product ID.
- Created the add-product and edit-product workflows.
- Shared form behavior between the add and edit experiences.
- Added archive and restore actions.
- Added permanent deletion with a confirmation prompt.
- Added action-specific success messages.
- Updated the displayed products immediately after changes.

Hour 3-4 | Validation and interface redesign

During the fourth hour, I completed the form validation and redesigned the interface. I replaced the original square appearance with a modern sage, cream, and forest-green visual system while keeping the application's underlying functionality intact.

- Required a non-empty product name and valid category and status selections.
- Required the price to be greater than zero.
- Prevented letters and invalid formats in numeric fields.
- Required stock to be a non-negative whole number.
- Added clear field-specific validation messages.
- Formatted all prices with two decimal places.
- Added rounded cards, buttons, filters, and form controls.
- Improved typography, spacing, contrast, shadows, icons, and action alignment.

Hour 4-5 | Customer preview, images, and responsive behavior

During the fifth hour, I created the read-only customer preview and finished the primary responsive behavior. The customer preview uses the current product state while removing all management controls. I also added image uploading and an in-page image viewer.

- Added a one-click owner/customer view switch.
- Prevented product editing from the customer preview.
- Displayed active products and clearly labelled active out-of-stock products.
- Added product image uploads and an in-page enlarged image dialog.
- Prevented image preview from opening a blocked or blank page.
- Improved layouts for desktop, tablet, and mobile widths.
- Adjusted navigation, cards, forms, buttons, and typography for smaller screens.
- Corrected image positioning and scrolling behavior.

Hour 5-6 | Reliability states and automated testing

During the sixth hour, I added the required reliability states and created the Playwright end-to-end test suite. I tested the main workflows using both desktop and mobile Chromium configurations.

- Added loading indicators, request-error recovery, and empty-result feedback.
- Verified invalid-product handling.
- Added Playwright end-to-end testing for desktop and mobile.
- Tested the product listing and required fields.
- Tested combined search, category, status, and both price-sort directions.
- Tested validation, add, edit, archive, restore, and delete workflows.
- Tested owner/customer views, image handling, loading states, and request failures.

Approximately 6:00-6:30 | Final debugging and verification

The project went approximately 15-30 minutes beyond the six-hour target during final testing. This time was spent correcting problems uncovered by automated tests and physical mobile-device testing rather than adding unrelated features.

- Corrected inconsistent search-clearing behavior and synchronized the relevant search state.
- Corrected a Playwright expectation involving filtered result counts.
- Verified that out-of-stock products appeared correctly in customer preview.
- Resolved a Next.js development-origin issue that prevented scripts from loading on a physical phone.
- Rechecked action alignment, price formatting, image preview behavior, and responsive layouts.
- Ran the final TypeScript, lint, build, and automated test checks.

Condensed time record

Approximate time	Development work
0:00-1:00	Application routes, types, API/data integration, images, and initial product listing
1:00-2:00	Search, category and status filters, price sorting, result counts, and inventory views
2:00-3:00	Product details, add/edit forms, state updates, archive, restore, and delete
3:00-4:00	Form validation and complete visual redesign
4:00-5:00	Customer preview, image handling, and responsive layouts
5:00-6:00	Reliability states and Playwright test coverage
6:00-6:30	Debugging issues discovered during automated and physical-device testing

Estimated hands-on development and testing time	Approximately 6 hours and 30 minutes

Planning, environment setup, GitHub organization, screenshots, documentation, and the final requirements comparison were completed separately and are not included in that development estimate.